

Reviewer Pack — Herbrand-Disjunction Length of PHP Bounds Tree-Resolution Le...

Entry 9152edf3c7fc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 22:52:30 UTC

Herbrand-Disjunction Length of PHP Bounds Tree-Resolution Leaves

Entry ID: 9152edf3c7fc **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 22:52:30 UTC

1. Conjecture

Field A (mathematical branch): Bounded arithmetic via Herbrand’s theorem in the Cook-Reckhow-Krajicek correspondence: the Herbrand-disjunction length $H_k(F)$ of a refutable bounded formula F , computed as the minimum number of ground term-tuples $t_1, \dots, t_m$ needed so that the disjunction $\bigvee_i \neg \varphi(t_i)$ (with φ the existential matrix of F ’s Skolemization) is a propositional tautology under the natural CNF coding — a PV/S^1_2 invariant that, by Buss’s witnessing theorem, polynomially relates to PV-proof length but is rarely measured directly on small instances **Field B** (complexity object): Tree-like resolution leaf count $L_T(PHP_n^{n+1} \vdash \perp)$ of the pigeonhole principle CNF on $n+1$ pigeons and n holes, equivalently DPLL search-tree leaves on the standard PHP encoding

Statement:

For every n with $3 \leq n \leq 10$, the Herbrand-disjunction length $H(PHP_n^{n+1})$ — defined operationally as the minimum number m of pigeon-hole assignment tuples $(a_1, b_1), \dots, (a_m, b_m)$ such that the propositional disjunction $\bigvee_{i \leq m} (\exists \{a_i, b_i\} \wedge \exists \{a_i, b'_i\})$ with $b'_i \neq b_i$ covers every total injection candidate witnessed by Buss-style elimination — satisfies $\log_2(L_T(PHP_n^{n+1})) = (1 \pm 0.25) \cdot \log_2(H(PHP_n^{n+1})) + o(n)$. One pair $(n, \text{witnessed mismatch outside the } \pm 0.25 \text{ window with both quantities exactly computed})$ refutes the conjecture.

Rationale (proposer’s reasoning):

Cook-Reckhow-Krajicek tie PV/S^1_2 provability of UNSAT to short tree-Frege/resolution refutations via Herbrand witnessing, but no one has empirically measured the Herbrand-witness count side-by-side with DPLL leaf count on PHP, the canonical hard instance. If the bounded-arithmetic correspondence is tight at small scale, the two logs should track linearly; deviation would localize where Buss’s witnessing slack hides. PHP avoids relativization (oracle-uniform) and is the textbook target separation between PV-style and stronger theories.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ebe3f0a57930278f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each n in $3..10$, compute $L_T(PHP_n^{\{n+1\}})$ and $H(PHP_n^{\{n+1\}})$ exactly across 5 fixed clause-ordering seeds; take per- n median ratio $r(n)=\log_2(L_T)/\log_2(H)$. Conjecture is SUPPORTED iff $r(n)\in[0.75,1.25]$ for all 8 n on $\geq 4/5$ seeds; FALSIFIED iff any n has median $r(n)$ outside $[0.75,1.25]$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Herbrand disjunction length pigeonhole principle resolution - bounded arithmetic PV witnessing tree-like resolution PHP lower bound - Herbrand complexity propositional proof DPLL pigeonhole tree resolution leaves

Top relevant hits considered: - [<http://arxiv.org/abs/1608.07792v1>] A Generalized Resolution Proof Schema and the Pigeonhole Principle - [<http://arxiv.org/abs/1411.3304v1>] On the complexity of finding falsifying assignments for Herbrand disjunctions - [<http://arxiv.org/abs/cs/0304041v1>] $P \neq NP$, propositional proof complexity, and resolution lower bounds for the weak pigeonhole principle - [<http://arxiv.org/abs/1007.1875v2>] Lower Bounds for Quantum Oblivious Transfer - [<http://arxiv.org/abs/0906.0693v3>] An improved lower bound on the counterfeit coins problem - [<http://arxiv.org/abs/1801.08709v2>] Adaptive Lower Bound for Testing Monotonicity on the Line - [<http://arxiv.org/abs/cs/0209032v3>] Complexity Results on DPLL and Resolution - [s2:2207.11284] Towards the shortest DRAT proof of the Pigeonhole Principle

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def lex_dppll(cnf, assignment):
    def dfs():
        if not cnf:
            return True
        var = next((v for v in range(len(cnf)) if v not in assignment), None)
        if var is None:
            return False
```

```

    for val in [True, False]:
        new_assignment = assignment.copy()
        new_assignment[var] = val
        new_cnf = []
        for clause in cnf:
            if any(var == v and val == True or var == -v and val == False for v in clause):
                continue
            elif all(var == -v or var == v for v in clause):
                return False
            else:
                new_clause = [v for v in clause if v != var and v != -var]
                if new_clause:
                    new_cnf.append(new_clause)

        if dfs():
            return True
    return False

return dfs()

def herbrand_disjunction_length(cnf):
    n = len(cnf)
    variables = set()
    for clause in cnf:
        variables.update(abs(v) for v in clause)

    def covers(assignment):
        for clause in cnf:
            if all(var not in assignment or (var < 0 and assignment[var] == False) or (var > 0 and assignment[var] == True) for var in clause):
                return False
        return True

    min_cover = float('inf')
    for i in range(1 <= len(variables)):
        assignment = {v: bool(i & (1 <= j)) for j, v in enumerate(variables)}
        if covers(assignment):
            min_cover = min(min_cover, sum(1 for var, val in assignment.items() if val))

    return min_cover

def build_php(n):
    variables = set()
    clauses = []
    for i in range(n):
        for j in range(n + 1):
            variables.add(i * (n + 1) + j)
            variables.add(-(i * (n + 1) + j))

    for i in range(n):
        for j in range(n + 1):
            clauses.append([- (i * (n + 1) + j)])

    for i in range(n):
        for j in range(n + 1):
            for k in range(j + 1, n + 1):
                clauses.append([(i * (n + 1) + j), -(i * (n + 1) + k)])

```

```

    return list(clauses)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in range(3, 11):
        cnf = build_php(n)
        leaves = lex_dp11(cnf, {})
        H = herbrand_disjunction_length(cnf)
        ratio = math.log2(leaves) / math.log2(H)
        results.append((n, leaves, H, ratio))

    median_ratio = sorted(ratio for _, _, _, ratio in results)[len(results) // 2]
    conjecture_holds = 0.75 <= median_ratio <= 1.25
    counterexample = "" if conjecture_holds else f"n={results[0][0]}, L_T={results[0][1]}, H={results[0][2]}"

    return {
        "metric_name": "ratio",
        "metric_value": median_ratio,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    ratios = [result["metric_value"] for result in results if "metric_value" in result]
    support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

    if all(0.75 <= r <= 1.25 for r in ratios):
        print(f"RESULT: SUPPORTED mean={sum(ratios)/len(ratios)} std={math.sqrt(sum((r - sum(ratios)/len(ratios))**2)/len(ratios))}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"n={results[0][0]}, L_T={results[0][1]}, H={results[0][2]}\"")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_06300ale.py", line 110, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_06300ale.py", line 88, in run_trial
    leaves = lex_dp11(cnf, {})
            ^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_06300ale.py", line 42, in lex_dp11
    return dfs()
           ^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_06300ale.py", line 38, in dfs
    if dfs():
        ^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_06300ale.py", line 38, in dfs
    if dfs():
        ^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_06300ale.py", line 38, in dfs
    if dfs():
        ^^^^^
[Previous line repeated 993 more times]
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_06300ale.py", line 21, in dfs
    var = next((v for v in range(len(cnf)) if v not in assignment), None)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
RecursionError: maximum recursion depth exceeded

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a RecursionError in lex_dp11 before producing any RESULT line, so neither L_T nor H was computed for any n. Per Rule 2, a crashed test yields INCONCLUSIVE. | next: Rewrite lex_dp11 iteratively (or raise sys.setrecursionlimit and fix the variable-selection bug iterating over range(len(cnf)) instead of variable indices), then re-run for n=3..10 across the 5 fixed seeds.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	23059
2	preregistration	claude_max	opus	0	0	6994
3	novelty	claude_max	opus	0	0	3413
4	novelty	claude_max	opus	0	0	9355
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18421
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12498
7	judge	claude_max	opus	0	0	4975

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 78716 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/9152edf3c7fc.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9152edf3c7fc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9152edf3c7fc.tar.gz` (if generated)